

Aim: Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

```
import sys
import hashlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.exceptions import InvalidSignature

def generate_keys():
    key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    private_bytes = key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    )
    public_bytes = key.public_key().public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )
    with open("private.pem", "wb") as f:
        f.write(private_bytes)
    with open("public.pem", "wb") as f:
        f.write(public_bytes)
    print("Keys generated: private.pem, public.pem")

def sign(message):
    with open("private.pem", "rb") as f:
        key = serialization.load_pem_private_key(f.read(), password=None)

    digest = hashlib.sha256(message.encode()).digest()

    signature = key.sign(
        digest,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )

    with open("signature.bin", "wb") as f:
```

```
f.write(signature)

print("Message      :", message)
print("SHA256       :", digest.hex())
print("Signature    :", signature.hex())
print("Status       : SIGNED")

def verify(message):
    with open("public.pem", "rb") as f:
        key = serialization.load_pem_public_key(f.read())

    digest = hashlib.sha256(message.encode()).digest()

    with open("signature.bin", "rb") as f:
        signature = f.read()

    print("Message      :", message)
    print("SHA256       :", digest.hex())
    print("Signature    :", signature.hex())

    try:
        key.verify(
            signature,
            digest,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256()
        )
        print("Status      : VALID")
    except InvalidSignature:
        print("Status      : INVALID")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python rsa_signature_cli.py [genkeys|sign|verify] [message]")
        sys.exit(1)

    action = sys.argv[1]
```

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

```
if action == "genkeys":  
    generate_keys()  
elif action == "sign":  
    sign(sys.argv[2])  
elif action == "verify":  
    verify(sys.argv[2])  
else:  
    print("Invalid action")
```

```
def verify(message):  
    64     print("Status : VALID")  
    65 except InvalidSignature:  
    66     print("Status : INVALID")  
    67  
    68 if __name__ == "__main__":  
    69     if len(sys.argv) < 2:  
    70         print("Usage: python rsa_signature_cli.py [genkeys|sign|verify] [message]")  
    71         sys.exit(1)  
    72  
    73     action = sys.argv[1]  
    74  
    75     if action == "genkeys":  
    76         generate_keys()  
    77     elif action == "sign":  
    78         sign(sys.argv[2])  
    79     elif action == "verify":  
    80         verify(sys.argv[2])  
    81
```

```
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> cd 'c:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY'; & 'c:\Users\Carol Pillai\AppData\Local\Programs\Python\Python39\python.exe' 'c:\Users\Carol Pillai\vscode\extensions\ms-python.debugpy-2025.6.0-win32-x64\bundled\libs\debugpy\launcher' '38267' '-c' 'c:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY\Pratical4.py'  
Usage: python rsa_signature_cli.py [genkeys|sign|verify] [message]  
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python Pratical4.py sign "hello carol"  
Message : hello carol  
SHA256 : 26ebec74422cad2454e8f72079ec23ee9b33af61fd6271770893d47cf2c4072e  
Signature : 4d362879a2b7c3f3c929562537e82f8a6eac74c9b88efc630f6a318b579be0d149d1bae18f44a6c2dfbbe3a061c6ff1758ec1840d9548fec77e8e6888783245e121e9104ce9e95bb5e016df0bb824114042545749a5054f36a5190f85961d1522bf14f47276a3e2d61bcb01ca7e2bc702933b1ecadde60bee7e1d865f991b34c01e6ecef8d768cc5ee513cc042302835ab8c5701a0f643488f5dacc1c3c5996fee57ae50779a87efcd2000704c3a45f53be39151898f833e27062c6e40b2ea31147c888607ba0bea53f5c371d11d770b5cdfc640622660c8f32008d3926efaea545709d9f8e452ea492afc5e1341d9fddf3838eed2f7576052628f54  
Status : SIGNED  
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY> python Pratical4.py verify "hello sahil"  
Message : hello sahil  
SHA256 : cdbb2703e9509a9a77d7d8e84e3b5296831e08a4a59a0a79d887c7a3407d000c  
Signature : 4d362879a2b7c3f3c929562537e82f8a6eac74c9b88efc630f6a318b579be0d149d1bae18f44a6c2dfbbe3a061c6ff1758ec1840d9548fec77e8e6888783245e121e9104ce9e95bb5e016df0bb824114042545749a5054f36a5190f85961d1522bf14f47276a3e2d61bcb01ca7e2bc702933b1ecadde60bee7e1d865f991b34c01e6ecef8d768cc5ee513cc042302835ab8c5701a0f643488f5dacc1c3c5996fee57ae50779a87efcd2000704c3a45f53be39151898f833e27062c6e40b2ea31147c888607ba0bea53f5c371d11d770b5cdfc640622660c8f32008d3926efaea545709d9f8e452ea492afc5e1341d9fddf3838eed2f7576052628f54  
Status : INVALID  
PS C:\Users\Carol Pillai\Desktop\College\Sem 5\CYBER SECURITY>
```

Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages. GUI

```
import tkinter as tk
from tkinter import messagebox, scrolledtext
import hashlib

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.exceptions import InvalidSignature

BG = "#ffe4ec"
PANEL = "#ffc2d9"
BTN = "#ff85a2"
BTN_HOVER = "#ff5c8a"
TEXT = "#5c1a33"

def generate_keys():
    key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    private_bytes = key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    )
    public_bytes = key.public_key().public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )
    with open("private.pem", "wb") as f:
        f.write(private_bytes)
    with open("public.pem", "wb") as f:
        f.write(public_bytes)
    messagebox.showinfo("Keys Generated", "private.pem and public.pem created")

def sign_message():
    message = msg_entry.get("1.0", tk.END).strip()
    if not message:
        messagebox.showwarning("Missing Input", "Enter a message to sign")
        return
    try:
```

```
with open("private.pem", "rb") as f:
    key = serialization.load_pem_private_key(f.read(),
password=None)
    digest = hashlib.sha256(message.encode()).digest()
    signature = key.sign(
        digest,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    with open("signature.bin", "wb") as f:
        f.write(signature)
    result_box.delete("1.0", tk.END)
    result_box.insert(tk.END, f"SHA256: {digest.hex()}\n\n")
    result_box.insert(tk.END, f"Signature: {signature.hex()}\n\n")
    result_box.insert(tk.END, "Status: SIGNED", "signed")
    result_box.tag_config("signed", foreground="#c2185b", font=("Segoe
UI", 10, "bold"))
except FileNotFoundError:
    messagebox.showerror("Error", "Generate keys first")

def verify_message():
    message = msg_entry.get("1.0", tk.END).strip()
    if not message:
        messagebox.showwarning("Missing Input", "Enter a message to
verify")
    return
    try:
        with open("public.pem", "rb") as f:
            key = serialization.load_pem_public_key(f.read())
        with open("signature.bin", "rb") as f:
            signature = f.read()
        digest = hashlib.sha256(message.encode()).digest()

        result_box.delete("1.0", tk.END)
        result_box.insert(tk.END, f"SHA256: {digest.hex()}\n\n")
        result_box.insert(tk.END, f"Signature: {signature.hex()}\n\n")

    try:
```

```
        key.verify(
            signature,
            digest,
            padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
salt_length=padding.PSS.MAX_LENGTH),
            hashes.SHA256()
        )
        result_box.insert(tk.END, "Status: VALID", "valid")
        result_box.tag_config("valid", foreground="#2e7d32",
font=("Segoe UI", 10, "bold"))
    except InvalidSignature:
        result_box.insert(tk.END, "Status: INVALID", "invalid")
        result_box.tag_config("invalid", foreground="#c62828",
font=("Segoe UI", 10, "bold"))
    except FileNotFoundError:
        messagebox.showerror("Error", "Sign a message first")

root = tk.Tk()
root.title("RSA Digital Signature")
root.geometry("500x480")
root.configure(bg=BG)

tk.Label(root, text="RSA Digital Signature Tool", bg=BG, fg=TEXT,
font=("Segoe UI", 16, "bold")).pack(pady=15)

tk.Button(root, text="Generate Keys", command=generate_keys, bg=BTN,
fg="white",
activebackground=BTN_HOVER, font=("Segoe UI", 11, "bold"),
relief="flat", padx=10, pady=8, width=20).pack(pady=10)

tk.Label(root, text="Message", bg=BG, fg=TEXT, font=("Segoe UI", 11,
"bold")).pack()
msg_entry = tk.Text(root, height=4, width=45, bg=PANEL, fg=TEXT,
font=("Segoe UI", 10), relief="flat")
msg_entry.pack(pady=8)

btn_frame = tk.Frame(root, bg=BG)
btn_frame.pack(pady=5)
```

```
tk.Button(btn_frame, text="Sign", command=sign_message, bg=BTN,
fg="white",
          activebackground=BTN_HOVER, font=("Segoe UI", 11, "bold"),
          relief="flat", padx=20, pady=8).grid(row=0, column=0, padx=10)

tk.Button(btn_frame, text="Verify", command=verify_message, bg=BTN,
fg="white",
          activebackground=BTN_HOVER, font=("Segoe UI", 11, "bold"),
          relief="flat", padx=20, pady=8).grid(row=0, column=1, padx=10)

tk.Label(root, text="Signature", bg=BG, fg=TEXT, font=("Segoe UI", 11,
"bold")).pack(pady=(15, 0))
result_box = scrolledtext.ScrolledText(root, height=5, width=45, bg=PANEL,
fg=TEXT,
                                     font=("Segoe UI", 9),
          relief="flat")
result_box.pack(pady=8)

root.mainloop()
```

Sheth L.U.J & Sir M.V College
Subject: Cyber & Information Security

